CODE REVIEW

Gridline GP: refactoring review

19 October 2026 · branch `foundation` at `e8df37f` · game (`src/`, ~387 KB), desktop shell, tools, and `phone-wheel/`

Verdict

The codebase is in good shape for its size. The simulation is pure (no DOM, runs in Node) and covered by 68 tests, tunables mostly live in data tables, and there are no import cycles. Lint is clean, and `tools/sim-trace.mjs` can prove that a refactor left the physics bit-identical.

The refactoring below makes the next features (car setup screen, phone-wheel merge, smarter AI, replays) cheaper and safer. None of it fixes a current bug, and none of it should change gameplay.

Findings

1 The car object has no defined shape

HIGH VALUE · MEDIUM EFFORT

Evidence. `createCar` declares about 60 fields, but 17 more are added later from other modules:

- `pit.js`: fuel, tyreWear, damage, pit, pitStops, warned, pullSide, pitLimiter
- `car.js`, `race.js`, `driving.js`: newWallHit, wallFront, cutWide, steerIn, dirtyK, beachClock

The game also writes 6 render fields (`px/pz/ph/rx/rz/rh`) into simulation objects from `main.js`.

Risk. A misspelt field silently creates a new one, and nothing documents what a car contains.

Fix.

- Declare every field in `createCar`, grouped by owner; each module supplies its own initial block, e.g. `pitState()`.
- Keep render state in a game-side `Map`.
- Add `// @ts-check` with JSDoc types to `src/sim`, checked by `tsc --noEmit` in `npm run check`. This catches field typos without converting to TypeScript.

2 Events carry their details on the car, not in the event

HIGH VALUE · SMALL–MEDIUM EFFORT

Evidence. Events are bare strings, and their details are read from car fields:

- `EV.WALL` uses `c.impact` and `c.impactHard`
- `EV.DAMAGE` uses `c.damageNote`
- `EV.CUT` uses `c.cutWide`
- `EV.PIT_STOP` uses `c.pit.service`

This only works because the game reads events in the same step they are created.

Fix. Push `{ type, ...details }` objects. The side-channel fields go away, each event explains itself, and logging and replays become possible.

3 `main.js` does too many jobs

MEDIUM VALUE · MEDIUM EFFORT

Evidence. 29 KB, 25 imports. It holds:

- loading, validating and saving settings
- assist presets and grid building
- about 25 hard-coded radio and toast messages
- the settings-screen wiring and all button wiring
- the frame loop, track switching and the debug API

Fix. Split it into:

- `setup-store.js`: pure functions (load, validate, save, `assistsOf`, `raceFormat`, `buildField`) that can be unit-tested
- `race-messages.js`: an event-to-message table
- `settings-binding.js`
- `debug.js`

`main.js` keeps the state machine and the loop.

4 Option lists are written out in several places

MEDIUM VALUE · SMALL EFFORT

Evidence. `'off'/'normal'/'high'`, `'auto'/'semi'/'manual'` and the race types appear separately in `loadSetup` validation, the Settings screen, `describeSetup` and `pit.js`.

Risk. A new option missed in one place resets the player's saved choice on load, with no error.

Fix. One `OPTIONS` table in `config.js`, read by the validation, the settings segments and the descriptions.

5 Tunable numbers are hard-coded in `pit.js`

, against the project's "tunables in tables" rule

MEDIUM VALUE SMALL EFFORT

Evidence. In `pit.js`:

- a 62 s lap-time guess
- a 0.75 tyre warning, duplicating `PIT.ai.tyres`
- fuel warnings at 1.1 and 1.6 laps
- the entry check `HW + 2.5`
- box braking 6 and accelerations 8/18
- garage spacing `-67.5 + 15k`
- lane edge 1.5

Smaller cases elsewhere: `car.js` (wall clearance 1.3, damping 0.3) and `collide.js` (1.2 m nose).

Fix. Move them into `PIT`, `WEAR`, `BARRIER` and `CONTACT`.

6 Small helpers are copied around

MEDIUM VALUE SMALL EFFORT

Evidence.

- The track-index wrap `(i % N + N) % N` is written 22 times across 11 files, with three local `at` helpers.
- Car speed `Math.hypot(v, vy)` is computed 8 times in 5 files.
- A smoothstep is defined inline in `pit.js`.

Fix. `wrapIdx(N, i)`, `smoothstep` and `speedOf(c)` in `math.js`.

7 Randomness is not seedable

MEDIUM VALUE SMALL EFFORT

Evidence. The simulation calls `Math.random` directly (line bias in `createCar`, AI mistakes in `driving.js`). Tests make it repeatable by swapping the global.

Fix. Give each race its own random generator (`race.rng`, seeded in `createRace`). Races become repeatable, which replays, ghost laps and the parked "rivals learn from you" plan all need.

8 Phone wheel: `server.py` is one 944-line file

MEDIUM VALUE MEDIUM EFFORT

Evidence. One file holds three output backends, certificates, pairing, three connection links (USB, Bluetooth, Wi-Fi), two web apps and the command line. There are no Python tests.

Fix.

- Split it into a package: `outputs`, `links`, `certs`, `web`, `main`.
- Add pytest for the pure parts.

Decide this together with the planned merge into the app: that merge probably means rewriting the server in Node inside Electron, which would replace this file.

9 Two copies of the phone wheel

HOUSEKEEPING

`~/Downloads/Phone-wheel-main` still runs the Wi-Fi server, and its GitHub workflow does not run from the `phone-wheel/` subfolder. Once `phone-wheel/` is the one in use, retire the old checkout so that changes don't land in the wrong copy.

Lower priority

ITEM	NOTE
Test tooling inside the shipping app	The smoke test and track shots live in `desktop/main.cjs`, the app's real entry point. Move them to `tools/`, like `car-shot.cjs`.
Game modules do work on import	They create the renderer and look up DOM elements as soon as they load, so game code can't be unit-tested. Items 3 and 4 move the logic most worth testing out of them.
Dense code style	Many lines hold several statements, some past 200 characters. Don't reformat everything (history noise); an ESLint `max-len` of about 160 for new code is enough.
Three.js r128 (2021)	It works, and upgrading is large for little immediate gain. Revisit before the visual polish stage.
`CLAUDE.md`, 330 dense lines	It mixes architecture with history. Split it into `docs/architecture.md` and a changelog.

Keep as is

- The pure simulation driven by tables
- The fixed 120 Hz step with render interpolation
- The single-file build
- The design and HUD tools
- `sim-trace`, which makes the refactors above provably safe

Plan

1. **Step 1:** items 6, 5 and 4 (small and mechanical; each checked with `sim-trace` and the tests)
2. **Step 2:** items 1 and 2 (car shape and events, with `@ts-check`)
3. **Step 3:** items 3 and 7
4. **Step 4:** item 8, decided with the phone-wheel merge

About two to three focused sessions in total. There should be no gameplay change; every step is checked by the tests and `sim-trace`.
